

A Continual Offline Reinforcement Learning Benchmark for Navigation Tasks

Anonymous^{1,2}

¹ Anonymous Structure A

² Anonymous Structure B

anonymous.mail@domain.com

Abstract—In dynamic domains such as autonomous robotics and video game simulations, agents must continuously adapt to new tasks while retaining previously acquired skills. This ongoing process, known as Continual Reinforcement Learning, presents several challenges, including the risk of forgetting previously acquired knowledge and the need for scalable solutions. Building on recent advances, we introduce a benchmark providing a suite of video-game navigation scenarios, thus filling a gap in the literature and capturing key challenges: catastrophic forgetting, task adaptation, and memory efficiency. We define a set of various tasks and datasets, evaluation protocols, and metrics to assess the performance of algorithms, including state-of-the-art baselines. Our benchmark is designed not only to foster reproducible research and to accelerate progress in continual reinforcement learning for gaming, but also to provide an evaluation framework for navigation methods and algorithms — helping practitioners to identify effective approaches to drive advances in production pipelines. <https://sites.google.com/view/continual-nav-bench/>

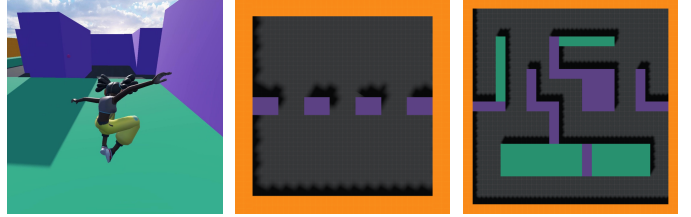
Index Terms—Deep Learning, Offline Reinforcement Learning, Continual Learning, Bots, Benchmarks, Human Generated Data.

I. INTRODUCTION

Humans continuously acquire new skills and may adapt to ever-changing world settings, seamlessly integrating fresh experiences with prior knowledge. In contrast, conventional Reinforcement Learning (RL) (1; 2) relies on extensive interactions and often suffers from catastrophic forgetting when agents are faced with sequential tasks. Continual Reinforcement Learning (CRL) (3; 4) addresses these limitations by enabling agents to learn incrementally, thus approximating the lifelong learning exhibited by humans.

Within CRL, Goal-Conditioned RL (GCRL) (5; 6) has garnered significant attention for its ability to condition policies on desired goal states — a feature particularly interesting for navigation tasks in robotics and video games. Modern video game environments, with their complex, dynamic, and sometimes stochastic layouts, demand bots¹ that not only adapt to new challenges but also retain previously acquired navigation strategies. Moreover, in production pipelines, it is essential to rigorously test navigation methods to determine which approaches yield the best performance in terms of both effectiveness and computational efficiency.

¹In a video game, a bot — short for robot — refers to any character operated by a computer program to simulate and, possibly, interact with a human player.



(A) A human playing. (B) $20\text{ m} \times 20\text{ m}$ map. (C) $60\text{ m} \times 60\text{ m}$ map.

Fig. 1: Visualization of an environment and a human playing : (a) **Human Player Interaction.** A third-person perspective of a human playing within a large maze ; (b) **Overview of a Small Map.** A top-down view of a small maze, showcasing the simple layout ; (c) **Overview of a Large Map.** A top-down view of a larger maze, highlighting a complex layout requiring better planning and actions.

The Offline RL (ORL) setting (7; 8) is appealing as it leverages large datasets of pre-recorded gameplay or simulation traces, bypassing the cost and risk of live data collection. However, despite ORL’s progress, standardized benchmarks for CRL in video game-inspired navigation tasks remain scarce. Current benchmarks (9; 10) rarely address the flexibility and memory requirements of production pipelines.

In this work, we introduce **Continual NavBench**, a novel benchmark for Continual Offline Reinforcement Learning focused on navigation in virtual game worlds. Continual NavBench bridges research and production needs by offering a suite of scenarios – including Godot engine (11) tailored made environments with human-generated data - for both research and real-world applications. Our benchmark includes :

- **Standardized Offline Datasets :** Sourced from 10 hours of human gameplay (around 3000 episodes) in diverse Godot mazes, to capture different navigation strategies.
- **Evaluation Protocols and Metrics :** We consider standard continual metrics – the *Performance*, the *Backward Transfer*, the *Forward Transfer*, the *Relative Model Size*, the *Training and Inference Costs* – to assess a model performance and efficiency, ensuring relevance for video game production.
- **Reproducible Benchmarking Tools :** Our open-source framework provides code, datasets, and evaluation protocols to ensure reproducible experiments and facilitate comparisons of methods for seamless integration into production pipelines.

II. RELATED WORK

Reinforcement Learning (RL) has achieved notable successes in domains such as robotics and video games (1; 2). In this regard, Benchmarks – including Atari 2600 (12), Procgen (13), and VizDoom (14) – have been used to evaluate RL agents in dynamic and rich environments. However, conventional RL typically relies on extensive online interaction and suffers from catastrophic forgetting when agents are faced with sequential tasks. Continual Reinforcement Learning (CRL) (3; 15) may address these issues by enabling agents to learn incrementally, better approximating the lifelong learning exhibited by humans.

In particular, Goal-Conditioned RL (GCRL) (5; 6) has shown promise for navigation tasks by conditioning policies on desired goal states. Despite these advances, most CRL research has focused on the online setting. In contrast, Offline CRL (7; 16) leverages fixed datasets of pre-recorded gameplay or simulation traces, bypassing the risks and costs of live data collection. Nevertheless, standardized benchmarks for Offline CRL in video game-inspired navigation tasks remain scarce. Existing benchmarks, such as Continual World (17) and CORA (18), primarily target online learning and rarely address production requirements like inference speed, memory efficiency, and model scalability.

Most CRL methods address the challenge of sequential task learning through various strategies : **Replay-Based** methods mitigate forgetting by storing past experiences and replaying them (19; 20), however they can be impractical due to significant storage requirements and potential privacy concerns, especially in industrial applications ; **Regularization** techniques such as Elastic Weight Consolidation (EWC) (21), or simply L2 regularization (22) introduce constraints on parameter updates, but they may struggle with highly diverse tasks. **Architectural** strategies, such as Progressive Neural Networks (PNNs) (23) or (24), modify a networks architecture to accommodate new tasks, nevertheless they can become memory-intensive and may lack scalability.

III. PRELIMINARIES

In this section, we present the theoretical background of CRL. These foundational concepts provide the basis for understanding and comparing the performance of different approaches.

A. Reinforcement Learning

We consider a Markov Decision Process (MDP) framework $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}_{\mathcal{S}}, \mathcal{P}_{\mathcal{S}}^{(0)}, \mathcal{R}, \gamma)$, which provides a formal framework for RL, where $\mathcal{S}$ is a state space, $\mathcal{A}$ an action space, $\mathcal{P}_{\mathcal{S}} : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ a transition function, $\mathcal{P}_{\mathcal{S}}^{(0)} \in \Delta(\mathcal{S})$ an initial distribution over the states, $\mathcal{R} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ a deterministic reward function, and $\gamma \in [0, 1]$ a discount factor. An agent’s behavior follows a policy $\pi_{\theta} : \mathcal{S} \rightarrow \Delta(\mathcal{A})$, parameterized by $\theta \in \Theta$. The objective is to learn optimal parameters $\theta_{\mathcal{M}}^*$ maximizing the expected cumulative reward $J_{\mathcal{M}}(\theta)$ or the success rate $\sigma_{\mathcal{M}}(\theta)$.

B. Offline Goal-Conditioned Reinforcement Learning

We extend the MDP to include a goal space $\mathcal{G}$, $\mathcal{P}_{\mathcal{S}, \mathcal{G}}^{(0)}$ an initial state-goal distribution, $\phi : \mathcal{S} \rightarrow \mathcal{G}$ a function mapping each state to the goal it represents, and $d : \mathcal{G} \times \mathcal{G} \rightarrow \mathbb{R}^+$ a distance metric on $\mathcal{G}$. Then, the policy $\pi_{\theta} : \mathcal{S} \times \mathcal{G} \rightarrow \Delta(\mathcal{A})$ and the reward function $\mathcal{R} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \times \mathcal{G} \rightarrow \mathbb{R}$ now depend on a goal $g \in \mathcal{G}$. We consider sparse rewards allocated when agents a goal : $\mathcal{R}(s_t, a_t, s_{t+1}, g) = \mathbb{1}(d(\phi(s_{t+1}), g) \leq \epsilon)$. Hence, given a pre-collected dataset $\mathcal{D} = \{(s, a, r, s', g)\}$ (7; 25), the policy loss is then optimized to reach all the specified goals.

C. Continual Reinforcement Learning

In CRL, an agent follows a sequence of tasks, or stream, $\mathcal{T} = (T_1, \dots, T_N)$, with $T_k = (\mathcal{M}_k, \mathcal{D}_k)$. We note as θ_k the parameters of a policy after learning on the k -th task. As the agent learns new skills, it must preserve (to prevent forgetting) and enhance (to encourage backward transfer) its performance on tasks already learned, while ideally having a relatively low number of parameters. To quantitatively compare CRL methods, we adopt standard metrics commonly used in the literature (15; 26) : **Performance (PER)** : $\frac{1}{N} \sum_{k=1}^N \sigma_{\mathcal{M}_k}(\theta_N)$; **Backward Transfer (BWT)** : $\frac{1}{N} \sum_{k=1}^N (\sigma_{\mathcal{M}_k}(\theta_N) - \sigma_{\mathcal{M}_k}(\theta_k))$; **Forward Transfer (FWT)** : $\frac{1}{N} \sum_{k=1}^N (\sigma_{\mathcal{M}_k}(\theta_k) - \sigma_{\mathcal{M}_k}(\tilde{\theta}_k))$; **Relative Model Size (MEM)** : $\frac{|\theta_N|}{|\theta_{\text{ref}}|}$; **Model Inference Cost (INF)**; **Model Training Cost (TRN)**. **PER** measures the success rate across all tasks. **BWT** indicates how learning a new task affects previous ones, while **FWT** measures the transfer of knowledge, using $\tilde{\theta}_k$ as randomly initialized parameters. **MEM** compares the memory load of the model to a reference one associated to parameters θ_{ref} . **INF** is the duration required for policy inference across tasks, and **TRN** is the overall training duration.

IV. ENVIRONMENTS, TASKS & DATASETS

A. Navigation Environments & Mazes

We propose video game-inspired navigation environments, which offer easy to use 3D maze configurations via our provided open-source code². Our benchmark comprises two families of mazes designed to evaluate CRL agents : **SimpleTown**, a collection of 8 relatively simple mazes (20 $m \times 20 m$), and **AmazeVille**, a more complex set of 8 mazes (60 $m \times 60 m$).

Action	Dimensions	Type	Human Controls / Keys
Move Forwards	1	bool	W
Move Backwards	1	bool	S
Move Left	1	bool	A
Move Right	1	bool	D
Run	1	bool	Shift
Jump	1	bool	Space
Turn	1	float	Mouse

TABLE I: **Available Action Keys.** This table details the control commands available to the agents in our environments. Actions include directional movement, running, jumping, and turning. The *Keys* indicate the human input used during data collection.

²GitHub Repository Link.

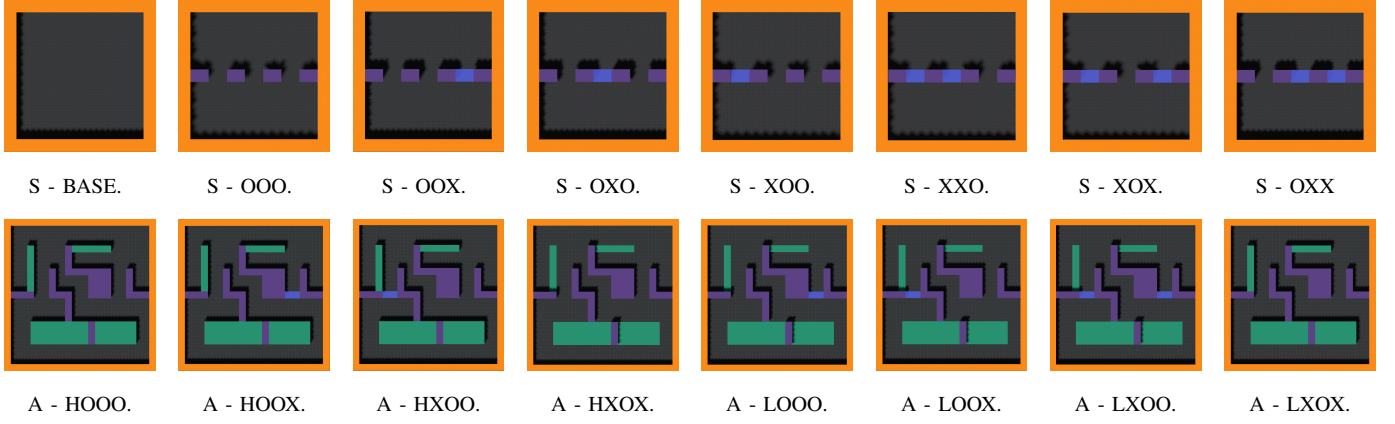


Fig. 2: **SimpleTown** mazes are relatively simple, with a size of 20×20 meters. The starting positions are randomly sampled on one side, and the goal positions are on the other side ; **AmazeVille** mazes, of 60×60 meters, are more challenging. They have a finite set of start and goal positions, with two subsets of maps : some with high blocks, *i.e.* not jumpable obstacles ; others with low blocks, *i.e.* jumpable ones. The naming convention of the tasks use a prefix for the maze family (S for SimpleTown, and A for AmazeVille), the subsequent characters encode key layout features where “O” and “X” are respectively open and closed doors, and “H” and “L” denote high and low blocks.

Feature	Size	Type	Feature	Size	Type
Position	3	float	Orientation	3	float
Goal Position	3	float	Velocity	3	float
RGB Image	(3, 64, 64)	float	Depth Image	(11, 11)	float
Floor Contact	1	bool	Wall Contact	1	bool
Goal Contact	1	bool	Timestep	1	int
Up Direction	3	float	-	-	-

TABLE II: **Available observation features.** Position data are in m , while the Orientation is in rad , and the Velocity in $m \cdot s^{-1}$. The RGB Images offer a visualization of the agent’s viewpoint, and the Depth Image is generated from 11×11 raycasts thrown towards nearest obstacles. Although our experiments focus on position and raycasts inputs, we also provide the pixel-based observations with all datasets.

B. Task Streams

Our benchmark defines task streams with different maze configurations. We distinguish two types of streams :

Random Streams – In AmazeVille we have :

Stream 1 : A-LOOX $\rightarrow$ A-HXOX $\rightarrow$ A-LXOX $\rightarrow$ A-HXOX ;

Stream 2 : A-HXOO $\rightarrow$ A-HOOX $\rightarrow$ A-LOOX $\rightarrow$ A-LXOO.

Topological Streams – Designed with systematic common changes in maze structure. We have in both environments :

Stream 1 : A-HOOX $\rightarrow$ A-HXOX $\rightarrow$ A-HXOX $\rightarrow$ A-HOOX ;

Stream 2 : A-LOOO $\rightarrow$ A-LOOX $\rightarrow$ A-LXOX $\rightarrow$ A-LXOO ;

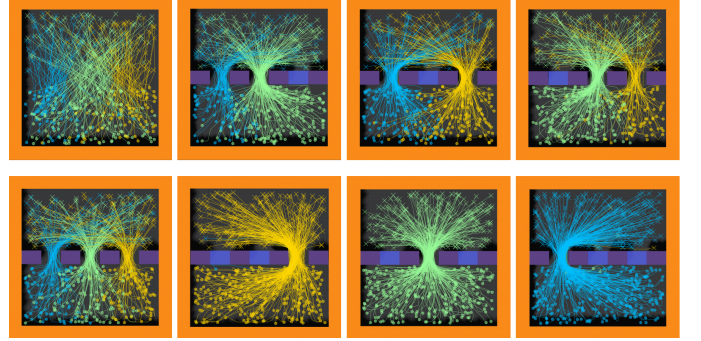
Stream 3 : S-BASE $\rightarrow$ S-OXO $\rightarrow$ S-BASE $\rightarrow$ S-OOX ;

Stream 4 : S-BASE $\rightarrow$ S-OXX $\rightarrow$ S-XOO $\rightarrow$ S-OXX.

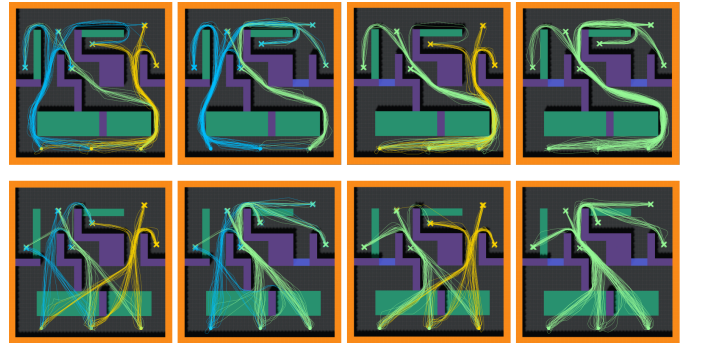
Notably, some tasks reoccur within a stream, offering an opportunity to evaluate whether an algorithm can recognize and reuse previously learned strategies without increasing model size or compromising performance. We encourage future users to tailor task streams to address specific research or production goals. Additionally, these environments are well-suited for developing and benchmarking GCRL algorithms.

C. Human Generated Datasets

We gathered 2800 trajectories over 10 hours of human plays, with 250 episodes per maze in SimpleTown and 100 episodes in AmazeVille. These datasets capture strategies that are valuable for training bots to exhibit human-like behavior.



(A) Trajectories generated within SimpleTown mazes.



(B) Trajectories generated within AmazeVille mazes.

Fig. 3: Visualization of the generated trajectories

To further support the research community, we open source the data alongside our benchmark framework. This ensures that other practitioners can readily access, reproduce, and build upon our work, fostering collaborative advancements in CRL, and more generally GCRL, for video game bots development.

V. BASELINE DETAILS

A. Hierarchical Imitation Learning as Backbone Algorithm

We consider Hierarchical Imitation Learning (HGCB) to address GCRL by decomposing tasks into more manageable ones. Given a MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}_{\mathcal{S}}, \mathcal{P}_{\mathcal{S},g^{(0)}}, \mathcal{R}, \gamma, \mathcal{G}, \phi, d)$ and a pre-collected dataset $\mathcal{D} = \{(s_t^i, a_t^i, r_t^i, s_{t+1}^i, g^i)\}$, the end-to-end policy $\pi_{\theta}(s, g)$ is split into two components :

- A **High-Level Policy** $\pi_{\theta_h}^h(s, g)$, which selects intermediate sub-goals that serve as waypoints towards the final goal ;
- A **Low-Level Policy** $\pi_{\theta_l}^l(s, g)$, which generates actions to move the agent towards the provided sub-goal.

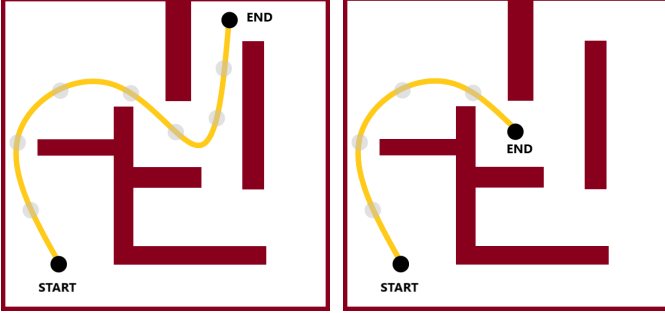
The high-level policy is optimized by minimizing :

$$\mathcal{L}_{\mathcal{D}}^h(\theta_h) = \mathbb{E}_{(s_t^i, s_{t+k}^i, g^i) \sim \mathcal{D}} \left[-\log(\pi_{\theta_h}^h(\phi(s_{t+k}^i) | s_t^i, g^i)) \right],$$

while the low-level policy minimizes :

$$\mathcal{L}_{\mathcal{D}}^l(\theta_l) = \mathbb{E}_{(s_t^i, a_t^i, s_{t+1}^i, s_{t+k}^i, g^i) \sim \mathcal{D}} \left[-\log(\pi_{\theta_l}^l(a_t^i | s_t^i, \phi(s_{t+k}^i))) \right].$$

Hence, the hyperparameter $k \in \mathbb{N}^*$ defines the temporal gap.



(A) Original Trajectory.

(B) Relabeled Trajectory.

Fig. 4: **Hindsight Experience Replay (HER) Illustration.**

Figure 4 illustrates *Hindsight Experience Replay (HER)* as a data augmentation strategy, leveraging the hierarchical structure. By relabeling transitions with alternative sub-goals, HER can enrich the dataset, especially in low or sparse data regimes.

VI. BASELINE DETAILS

A. Continual Reinforcement Learning Baselines

We evaluate a range of baseline methods designed to learn from a task stream. These methods span several categories :

Naive Methods :

Naive Learning Strategy or From Scratch (SC1 & SCN) : in SC1, a single policy is learned from the latest dataset and then applied unchanged to all tasks. In SCN, a new policy is trained for each task, improving performance at the cost of a memory load. **Freeze Strategy (FZ) :** here, a single policy is trained on the first task and applied without modification to all subsequent tasks. **Finetuning Strategy (FT1 & FTN) :** the Finetuning Strategy involves adapting a policy learned from the initial task to each subsequent task, either by continuously updating a single policy (FT1) or by copying and then updating the policy for each new task (FTN), allowing for a better adaptation.

Replay-Based Method :

Experience Replay Strategy (RPL) : this method aggregates all task datasets, up to the current one, into a single replay buffer and trains a single policy on the combined data, ensuring continuous exposure to earlier experiences. Such a process may allow to avoid catastrophic forgetting, and eventually build common skills and knowledge across tasks.

Weight Regularization Methods :

Elastic Weight Consolidation (EWC) (21) : this strategy mitigates catastrophic forgetting by selectively slowing down learning on certain weights based on their importance to previously learned tasks. This importance is measured by the Fisher Information Matrix, which quantifies the sensitivity of the output function to changes in the parameters. EWC introduces a quadratic penalty, constraining the parameters close to their values from previous tasks, where the strength of the penalty is proportional to each parameter's importance. **L2-Regularization Finetuning (L2) (22) :** This strategy also mitigates catastrophic forgetting by adding an L2 penalty to the loss, discouraging large weight changes during training. This helps preserve knowledge from previous tasks by promoting stability in the learned representations. As with EWC, L2-regularization struggles in CRL for navigation tasks, especially when actions or dynamics change drastically. The method limits the network's flexibility by forcing small weight updates, making it difficult to adapt to tasks that require distinct actions for similar states, which is critical in evolving environments.

Architectural Methods :

Progressive Neural Networks (PNN) (23) : This framework introduces a new column layers for each task, freezing previous weights to preserve knowledge. Lateral connections allow feature transfer, leveraging prior experience while avoiding interference. By integrating previously learned features, PNN builds richer compositional representations, ensuring that prior knowledge is preserved and used throughout the feature hierarchy. Nevertheless the model grows with each task, limiting scalability for long streams tasks or limited memory contexts. **Hierarchical Subspace of Policies (HiSPO) (24):** this strategy partitions the model into distinct subspaces of neural networks (27) for high-level and low-level control. Initially, anchor parameters for both subspaces are trained on the first task to form a foundational knowledge base. For each subsequent task, new anchors are introduced and optimized using the task's dataset. The algorithm then samples anchor weight configurations to explore the previous subspace, and it selects the configuration that minimizes the loss. Finally, by comparing the loss of the extended subspace with that of the previous configuration, under a threshold ϵ , HiSPO either prunes the new anchor or retains it. This process integrates new information while preserving and reusing prior knowledge, ensuring scalability and adaptability across tasks.

B. Implementation Details

We based our networks (28) and hyperparameters on the approach described in (29). In all our experiments, we employ Residual MLPs with layer normalization applied to the hidden layers. Both the high-level and the low-level policy networks use 256 hidden units, and GELU layer activations.

We set the batch size to 64 and use a learning rate of 3×10^{-4} for both AmazeVille and SimpleTown. The sub-goal distance is defined with $k = 10$ way steps for AmazeVille and $k = 3$ for SimpleTown. Additionally, HER sampling temperatures are set to 100.0 for AmazeVille and 15.0 for SimpleTown.

For both EWC and L2, we evaluated five regularization strengths $\lambda \in \{1 \times 10^{-2}, 1 \times 10^{-1}, 1, 1 \times 10^1, 1 \times 10^2\}$ and selected the best-performing model for each stream. Similarly, for HiSPO, we experimented with acceptance thresholds $\epsilon_h, \epsilon_l \in \{5 \times 10^{-2}, 1 \times 10^{-1}\}$, to decide whether to prune or extend the subspace, and weights for the cosine similarity loss as $\lambda_h, \lambda_l \in \{5 \times 10^{-2}, 1 \times 10^{-1}\}$, for functional diversity.

VII. BENCHMARK STUDY

TODO.

VIII. DISCUSSION

TODO.

REFERENCES

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning : An Introduction, Second Edition*. MIT Press, 2018.
- [2] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, “Deep reinforcement learning : A brief survey,” *IEEE Signal Processing Magazine*, 2017.
- [3] K. Khetarpal, M. Riemer, I. Rish, and D. Precup, “Towards continual reinforcement learning : A review and perspectives,” *Journal of Artificial Intelligence Research*, 2022.
- [4] L. Wang, X. Zhang, H. Su, and J. Zhu, “A comprehensive survey of continual learning : Theory, method and application,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- [5] Y. Ding, C. Florensa, M. Phielipp, and P. Abbeel, “Goal-conditioned imitation learning,” *Advances in Neural Information Processing Systems*, 2019.
- [6] M. Liu, M. Zhu, and W. Zhang, “Goal-conditioned reinforcement learning : Problems and solutions,” *IJCAI*, 2022.
- [7] S. Levine, A. Kumar, G. Tucker, and J. Fu, “Offline reinforcement learning : Tutorial, review, and perspectives on open problems,” 2020.
- [8] A. Kobanda, C. Valliappan, J. Romoff, and L. Denoyer, “Learning computational efficient bots with costly features,” in *2023 IEEE Conference on Games (CoG)*, pp. 1–8, IEEE, 2023.
- [9] V. Volz and B. Naujoks, “Towards game-playing ai benchmarks via performance reporting standards,” in *2020 IEEE Conference on Games (CoG)*, pp. 764–771, IEEE, 2020.
- [10] G. Macaluso, A. Sestini, and A. D. Bagdanov, “A benchmark environment for offline reinforcement learning in racing games,” in *2024 IEEE Conference on Games (CoG)*, pp. 1–4, IEEE, 2024.
- [11] Godot, “Godot game engine,” 2020.
- [12] V. Kurin, S. Nowozin, K. Hofmann, L. Beyer, and B. Leibe, “The atari grand challenge dataset,” *arXiv preprint arXiv:1705.10998*, 2017.
- [13] K. Cobbe, C. Hesse, J. Hilton, and J. Schulman, “Leveraging procedural generation to benchmark reinforcement learning,” in *International conference on machine learning*, pp. 2048–2056, PMLR, 2020.
- [14] M. Kempka, M. Wydmuch, G. Runc, J. Toczek, and W. Jaśkowski, “Vizdoom: A doom-based ai research platform for visual reinforcement learning,” in *2016 IEEE conference on computational intelligence and games (CIG)*, pp. 1–8, IEEE, 2016.
- [15] N. Díaz-Rodríguez, V. Lomonaco, D. Filliat, and D. Maltoni, “Don’t forget, there is more than forgetting : new metrics for continual learning,” 2018.
- [16] J. Liu, W. Li, X. Yue, S. Zhang, C. Chen, and Z. Wang, “Continual offline reinforcement learning via diffusion-based dual generative replay,” *arxiv*, 2024.
- [17] M. Wolczyk, M. Zajac, R. Pascanu, L. Kucinski, and P. Milos, “Continual world : A robotic benchmark for continual reinforcement learning,” 2021.
- [18] S. Powers, E. Xing, E. Kolve, R. Mottaghi, and A. Gupta, “Cora : Benchmarks, baselines, and metrics as a platform for continual reinforcement learning agents,” in *Conference on Lifelong Learning Agents*, PMLR, 2022.
- [19] D. Rolnick, A. Ahuja, J. Schwarz, T. Lillicrap, and G. Wayne, “Experience replay for continual learning,” *Advances in neural information processing systems*, 2019.
- [20] Y. Huang, K. Xie, H. Bharadhwaj, and F. Shkurti, “Continual model-based reinforcement learning with hypernetworks,” in *International Conference on Robotics and Automation (ICRA)*, IEEE, 2021.
- [21] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, *et al.*, “Overcoming catastrophic forgetting in neural networks,” *Proceedings of the national academy of sciences*, 2017.
- [22] S. Kumar, H. Marklund, and B. V. Roy, “Maintaining plasticity in continual learning via regenerative regularization,” 2023.
- [23] A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell, “Progressive neural networks,” *arxiv preprint*, 2016.
- [24] A. Kobanda, R. Portelas, O.-A. Maillard, and L. Denoyer, “Hierarchical subspaces of policies for continual offline reinforcement learning,” *arXiv preprint arXiv:2412.14865*, 2024.

- [25] E. Biré, A. Kobanda, L. Denoyer, and R. Portelas, “Efficient active imitation learning with random network distillation,” *arXiv preprint arXiv:2411.01894*, 2024.
- [26] R. Kemker, M. McClure, A. Abitino, T. Hayes, and C. Kanan, “Measuring catastrophic forgetting in neural networks,” in *Proceedings of the AAAI conference on artificial intelligence*, 2018.
- [27] M. Wortsman, M. C. Horton, C. Guestrin, A. Farhadi, and M. Rastegari, “Learning neural network subspaces,” in *International Conference on Machine Learning*, 2021.
- [28] J. Ansel and al., “PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation,” in *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ACM, 2024.
- [29] D. Ghosh, C. A. Bhateja, and S. Levine, “Reinforcement learning from passive data via latent intentions,” in *International Conference on Machine Learning*, PMLR, 2023.